

Revised Location Statistic: June 10, 2026

I really like the Location statistic, and was not satisfied with its distribution, and wanted to improve it. To explore what could be done, I began by removing the exponent on the error. This produces something symmetric, so there should be of recovering the symmetry while still squaring the errors.

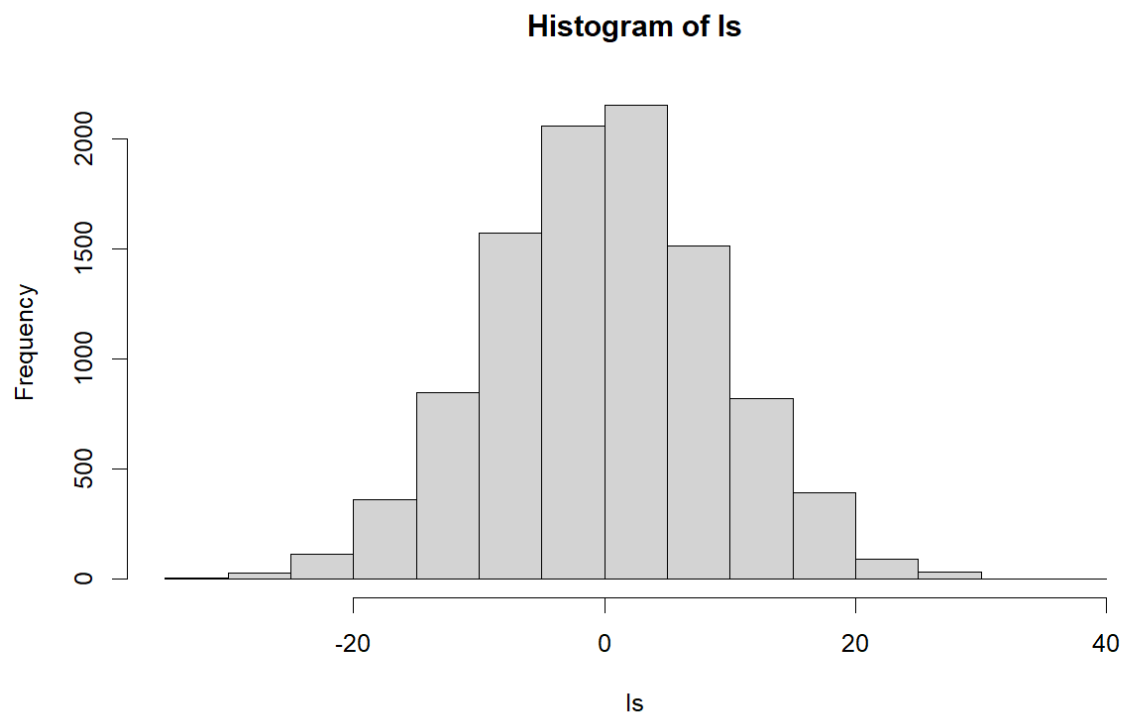
```
n <- 1000
reps <- 10000

ls <- numeric()
lvec <- (1:n)/(n+1)

set.seed(708)
for (r in 1:reps){
  pvec <- sort(runif(n))
  ls[[r]] <- sum(pvec-lvec)
}

print(mean(ls)) # >>> -0.06451141
print(sd(ls)) # >>> 9.13571
print(range(ls)) # >>> -33.43734 38.05093

silent <- hist(ls)
```



As recommended, I asked Gemini if it could generate a solution to this issue; one which maintains the direction while also squaring the errors. It suggested to calculate the bias of the errors to determine the direction, while still calculating the SSE. With these two values, the Location is calculated as shown below. The resulting distribution is symmetric, and is similar in structure (range) to the original approach. The entire Gemini chat is attached at the bottom of this document.

$$\text{Location} = \text{sign}\left(\sum_i e_i\right) \times \sum_i e_i^2$$

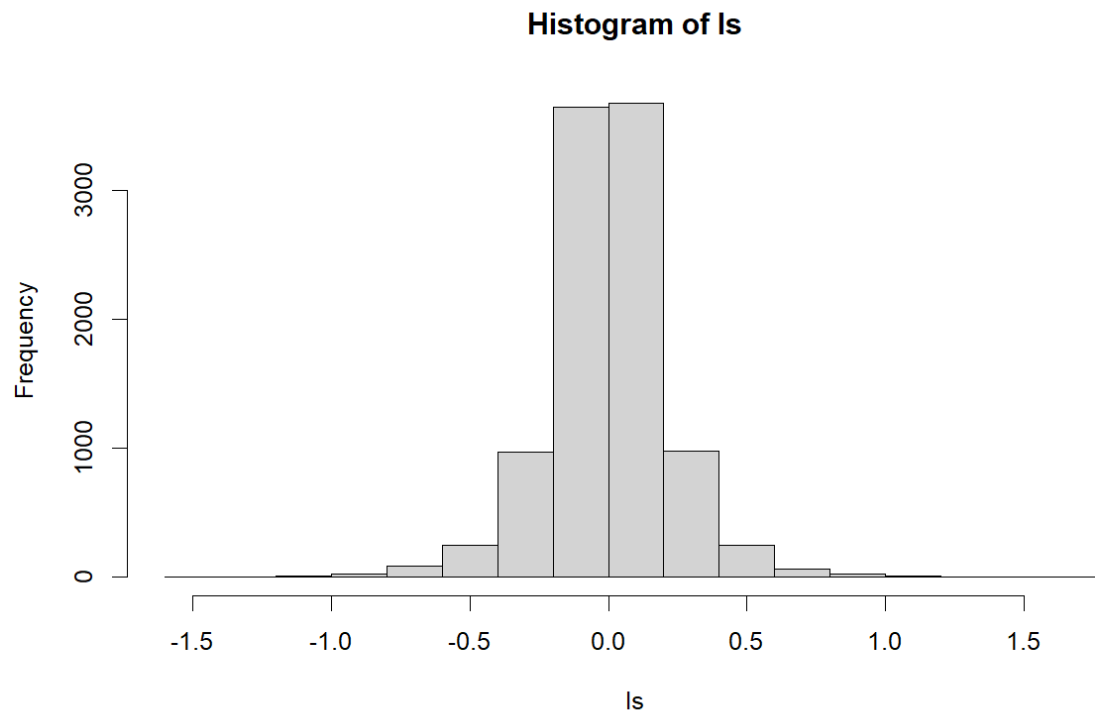
```
n <- 1000
reps <- 10000

ls <- numeric()
lvec <- (1:n)/(n+1)

set.seed(708)
for (r in 1:reps){
  pvec <- sort(runif(n))
  ls[[r]] <- sign(sum(pvec-lvec)) * sum((pvec-lvec)^2)
}

print(mean(ls)) # >>> -0.001432242
print(sd(ls)) # >>> 0.2240697
print(range(ls)) # >>> -1.454518 1.790883

silent <- hist(ls)
```



With the symmetry present, we can begin by scaling by the variance of each Uniform Order Statistic, which follows $U_{(i)} \sim \text{Beta}(i, n - i + 1)$, with variance $\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$. The scaled version still retains the symmetry.

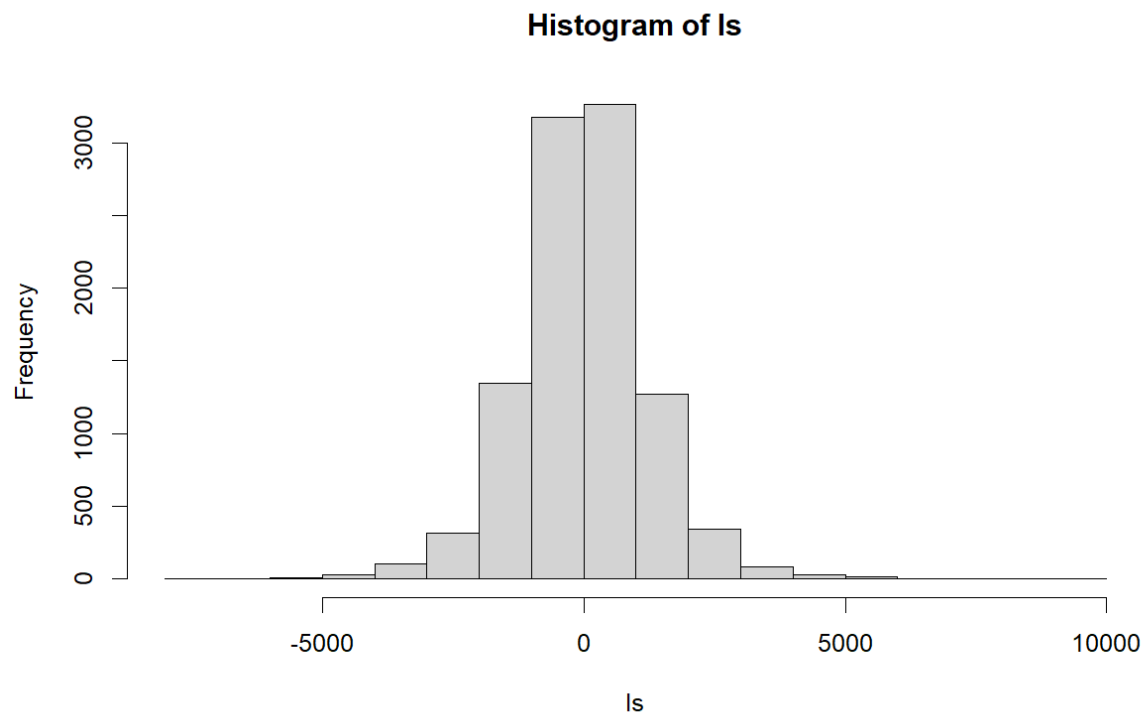
```
n <- 1000
reps <- 10000

ls <- numeric()
lvec <- (1:n)/(n+1)

set.seed(708)
for (r in 1:reps){
  pvec <- sort(runif(n))
  ls[[r]] <- sign(sum(pvec-lvec)) * sum((pvec-lvec)^2 / ((1:n)*(n - (1:n) + 1)/((n+1)^2 * (n+2))))
}

print(mean(ls)) # >>> -5.220444
print(sd(ls)) # >>> 1261.055
print(range(ls)) # >>> -7099.510 9175.432

silent <- hist(ls)
```



Gemini Prompts:

1. an issue with a sum of squared error statistic across many simulations is that the statistic will not be negative, and likely non-symmetric. If you use the sum of errors, without squaring, then you can maintain symmetry but then face the issue of positive and negative errors canceling within a single simulation. My question is whether there exists a way to maintain the ability of squaring to manage weights of larger errors along with some direction to keep it symmetric
2. but with option 1, if i have 100 samples in each simulation, a +4 and -4 will still cancel out when summing the squared error of themself
3. it seems that the “magnitude + bias” approach creates something symmetric - thanks

an issue with a sum of squared error statistic across many simulations is that the statistic will not be negative, and likely non-symmetric. If you use the sum of errors, without squaring, then you can maintain symmetry but then face

Yes, absolutely. You are hitting on a classic problem in statistics and forecasting: how to penalize large errors heavily (which squaring does beautifully) without losing the sign/direction of the error or letting positive and negative errors cancel each other out.

Several excellent techniques achieve exactly what you are looking for. Here are the best ways to maintain that "squared" penalty while preserving symmetry and direction.

1. The Signed Square Error (or "Directional" SSE)

The most direct answer to your question is to apply the squaring operation *only* to the magnitude of the error, and then reapply the original sign.

For a single simulation i with error e_i , the metric is defined as:

$$\text{Signed Error}_i = \text{sign}(e_i) \cdot e_i^2$$

Where $\text{sign}(e_i)$ is $+1$ if the error is positive, and -1 if it is negative.

- **Why it works:** If you have an error of $+4$, it becomes $+16$. If you have an error of -4 , it becomes -16 .
- **The Aggregation:** When you sum these across many simulations, positive and negative simulations **will not cancel out within a single simulation** (because you sum them *after* squaring). Across simulations, it will yield a symmetric distribution centered around zero if your model is unbiased.

2. Separate Overprediction and Underprediction Metrics

In many professional contexts (like supply chain or financial forecasting), treating positive and negative squared errors as a single aggregated pool muddies the water. Instead, you split them into two separate metrics before summing.

- **Sum of Squared Overpredictions (SSO):** Sum of e_i^2 only where $e_i > 0$.
- **Sum of Squared Underpredictions (SSU):** Sum of e_i^2 only where $e_i < 0$.

The Benefit: This completely preserves the "squared" penalty for large errors, prevents any cross-simulation cancellation, and gives you two perfectly clean,

non-negative metrics that you can compare. If $SSO > SSU$, you know your model has a specific direction of bias.

3. The Skewness / Asymmetry Framework

If your ultimate goal is to look at the *distribution* of these errors across many simulations and check for symmetry, you shouldn't just look at the sum. You should look at the **Skewness** of the errors.

If you calculate the standard **Mean Squared Error (MSE)** or **Root Mean Squared Error (RMSE)** for each individual simulation, you can then look at the distribution of those RMSEs across your thousands of simulations.

Because RMSE is strictly positive, its distribution across simulations will naturally be skewed right. To restore symmetry for statistical testing, you can apply a **Log Transformation**:

$$\log(\text{RMSE})$$

Transforming the squared metrics into log-space frequently normalizes the data, pulling in the extreme right tail and creating a symmetric, bell-shaped distribution across your simulation trials.

Summary Recommendation

- If you want a **single number** that shows if your simulations lean positive or negative while heavily penalizing big misses: Use the **Signed Square Error**.
- If you want to **diagnose your model's behavior**: Split your analysis into **Squared Overpredictions** and **Squared Underpredictions**.

Which of these approaches aligns closer to how you plan to use this statistic for your simulations?

but with option 1, if i have 100 samples in each simulation, a +4 and -4 will still cancel out when summing the squared error of themself

Ah, you are completely right. I misunderstood how you were aggregating. If you are summing *within* a single simulation first, then a +4 and a -4 inside that same simulation would cancel out to 0 ($16 - 16 = 0$), completely hiding the fact that the simulation actually had two large errors.

If your goal is to have **one representative number per simulation** that captures both the *magnitude* (with a squared penalty) and the *overall direction* (positive/negative bias) of that simulation, you need to decouple the magnitude from the direction during the aggregation step.

Here are two ways to fix this so internal errors don't cancel out:

1. The "Magnitude + Bias" Decomposition (Recommended)

Instead of trying to force a single number to do two opposing jobs, you calculate two distinct numbers for each simulation:

1. **The Magnitude (SSE):** Sum the squared errors normally. $\sum e_i^2$. Because everything is squared, nothing cancels out. This tells you how *accurate* the simulation was.
2. **The Direction (Bias):** Sum the raw errors. $\sum e_i$. This tells you if the simulation overshot or undershot on average.

To get your final metric for that simulation, you multiply the **total magnitude** by the **sign of the total bias**:

$$\text{Simulation Score} = \text{sign} \left(\sum e_i \right) \times \sum e_i^2$$

- **How it handles the +4 and -4 scenario:** The sum of squared errors is $16 + 16 = 32$. The net bias is $+4 + (-4) = 0$. Your score is $0 \times 32 = 0$. This correctly reflects that while there were errors, the simulation had absolutely zero directional bias.
- **How it handles a biased simulation (e.g., errors of +4 and +4):** SSE is $16 + 16 = 32$. Net bias is $+8$ (positive sign). Your score is $+32$.

Across many simulations, the distribution of these "Simulation Scores" will be symmetric around zero if your overall model is unbiased.

2. Root Mean Square Error (RMSE) with Signed Mean Error

This is a variation of the above approach used heavily in meteorology and climate modeling to evaluate simulations.

For each simulation, you calculate:

- **RMSE:** $\frac{1}{n} \sum e_i^2$ (Captures heavily-penalized variance/error magnitude)
- **Mean Error (ME):** $\frac{1}{n} \sum e_i$ (Captures the directional bias)

You then define the simulation's metric as:

$$\text{Directional RMSE} = \text{sign}(\text{ME}) \times \text{RMSE}$$

This gives you a metric that is in the same units as your original data, penalizes larger errors heavily via the RMSE calculation, but uses the overall net bias of the simulation to determine whether that simulation's line item gets a positive or negative sign.

3. The Power Transform ($e^{1.5}$ or e^3)

If you absolutely want a single mathematical function where you just plug in errors, sum them, and get a symmetric distribution without any conditional "sign switching" logic, you have to move away from even exponents (like squaring).

Even exponents destroy the negative sign, forcing us to use absolute values or sign functions. Odd exponents preserve the sign, but e^3 (cubing) might penalize large errors *too* severely for your needs.

An alternative is to use an exponent like 1.5 applied to the absolute value, then restoring the sign:

$$\text{Metric} = \sum \text{sign}(e_i) \cdot |e_i|^{1.5}$$

While not a "square," a 1.5 or 2.5 power still penalizes larger errors progressively more than smaller errors, preserves the sign naturally, and allows you to sum everything safely within a single simulation without $+4$ and -4 cancelling out (because $|4|^{1.5} = 8$ and $|-4|^{1.5} = 8$, and the signs make them $+8$ and -8). *Wait, actually, $+8$ and -8 still cancel out to 0 here!* You are completely right—mathematically, if a simulation has equal and opposite errors, any directional metric *should* cancel them out to show the net bias is zero. If you want to know that errors occurred *despite* them canceling out, you have to separate Magnitude from Bias as shown in Option 1 and 2.

Does the "Magnitude + Bias" decomposition in Option 1 capture what you are trying to visualize across your simulations?

it seems that the "magnitude + bias" approach creates something symmetric - thanks

You've got it. It elegantly solves the problem because it forces the "error penalization" and the "direction" to happen at two different levels of aggregation.

By calculating the total variance (the magnitude) first, you capture all the turbulence inside that simulation. By multiplying it by the overall sign of the bias, you cleanly map that simulation onto a negative or positive axis.

When you plot this "Magnitude + Bias" metric across thousands of simulations, you will get a beautiful, symmetric distribution (assuming your simulator doesn't have an inherent structural bias). The variance of that distribution will show you how volatile the simulations are, while the symmetry around zero will confirm if your system is balanced.

Good luck with the simulation analysis! If you run into any edge cases while coding this up, feel free to bounce them over.